

Série des exercices N°2

Exercice 1 :

Écrire une fonction **def AfficheSousSequences(T)** qui à partir d'un tableau d'entiers *T* affiche sur l'écran la liste des sous-séquences croissantes de ce tableau.

Exemple :

Soit *T* un tableau de 15 éléments : 1, 2, 5, 3, 12, 25, 13, 8, 4, 7, 24, 28, 32, 11, 14

Les séquences strictement croissantes qui seront affichées :

< 1, 2, 5 >; < 3, 12, 25 >; < 13 >; < 8 >; < 4, 7, 24, 28, 32 >; < 11, 14 >

Exercice 2: Dans cette partie, il s'agit de représenter des ensembles finis par des tableaux. On appellera "Ensemble Tableau", tout tableau de nombres entiers qui sont deux à deux différents.

- 1) Ecrire le code de la fonction : **def Intersection(T1, T2)** qui retourne l'ensemble tableau *T* contenant tous les éléments communs entre l'ensemble tableau *T1* et l'ensemble tableau *T2*. L'ensemble tableau *T* représente un ensemble et par conséquent tous ses éléments doivent être différents.

Exemples :

Si *T1*=[0, 3, 6, 8, 26, 19,33, 5, 11] et *T2*=[0, 3, 8, 16, 12,33] alors :*T*=[0, 3, 8, 33].

- 2) Soit *T* un ensemble tableau trié par ordre croissant dont les éléments sont tous dans l'intervalle [0, *N*] où *N* est une valeur entière qui représente le maximum du tableau.

Ecrire le code de la fonction **def Complement(T)** qui retourne le complément de *T* dans [0,max(*T*)], c'est-à-dire **les entiers** *x* de l'intervalle [0,max(*T*)] : *x* ∉ *T*. (*T* est trié)

- 3) Soit *T1* et *T2* deux ensembles tableaux triés par ordre croissant.

Ecrire le code de la fonction **def Fusion(T1,T2)** qui retourne l'ensemble tableau *T3*=*T1*∪*T2* obtenu par fusion de *T1* et *T2* de telle sorte que *T3* soit trié et ne contient pas de doublons.

Si *T1*=[3, 6, 8, 26, 109,133, 165, 181] et *T2*=[3, 8, 109, 112,303] alors :

T3=[3, 6,8, 26,109,112,133,165,181,303].

Exercice 3:

Liste des fréquences :

La première étape de la méthode de compression de Huffman consiste à compter le nombre d'occurrences de chaque élément de la liste des entiers.

Écrire une fonction : **frequences (L)** , qui reçoit en paramètre une liste d'entiers **L**, et qui retourne une liste de tuples : Chaque tuple est composé d'un élément de **L** et de son occurrence (répétition) dans **L**. Les premiers éléments des tuples de la liste des fréquences doivent être tous différents (sans doublon).

Exemple : $L = [12, 29, 46, 29, 31, 8, 12, 55, 29, 8, 12, 29, 31, 29, 8, 29, 8]$

La fonction **frequences (L)** retourne la liste $F = [(8, 4), (12, 3), (46, 1), (55, 1), (29, 6), (31, 2)]$

Exercice 4: Compression de données

La **compression de données** est une opération qui consiste à transformer une suite de bits **A** en une suite de bits **B** plus courte. La décompression est l'opération inverse de la compression.

I. Codage d'un nombre

Le **codage de Fibonacci** est une technique de compression de données qui utilise les nombres de la [suite de Fibonacci](#), dont chaque terme est la somme des deux termes consécutifs précédents. La suite de Fibonacci est définie comme suit:

$$U_0=1$$

$$U_1=1$$

$$U_n=U_{n-1}+U_{n-2}$$

Le principe du codage de Fibonacci est décrit par l'algorithme suivant :

Pour [encoder](#) un entier **X** :

- ✓ Créer un tableau avec 2 lignes.
- ✓ Dans **la première ligne** du tableau, mettre les chiffres de la suite de Fibonacci (1, 2, 3, 5, 8...) inférieurs ou égaux à **X**.
- ✓ Décomposer l'entier **X** en une somme d'entiers correspondant aux éléments de la ligne du tableau, en employant les plus grands possibles.
- ✓ Dans la **deuxième ligne** du tableau, mettre des « 1 » en dessous des éléments qui ont permis de décomposer **X**, « 0 » sinon.

Exemple

Décomposition de 50.

Les éléments de la ligne du tableau sont : 1 2 3 5 8 13 21 34

50 = 34 + 13 + 3 (50 = 34 + 8 + 5 + 3 est incorrect car le 13 n'a pas été utilisé) où le tableau :

Suite de Fibonacci	1	2	3	5	8	13	21	34
Présence dans la décomposition	0	0	1	0	0	1	0	1

Il reste à écrire le codage du nombre 50 : 00100101

Dans ce qui suit, on se propose d'étudier le **codage de Fibonacci en utilisant une liste à deux dimensions**.

Question 1:

Ecrire le code d'une fonction d'entête : **def fibo(n)** qui retourne le n-ième terme de la suite de Fibonacci.

Rappel : La suite de Fibonacci est définie comme suit:

$$U_0=1$$

$$U_1=1$$

$$U_n=U_{n-1}+U_{n-2} \text{ pour tout } n \geq 2$$

Question 2:

Écrire une fonction d'entête **def SuiteFibo(n)** qui retourne une liste à deux lignes, contenant dans la première ligne tous les termes de la suite de Fibonacci (1, 2, 3, 5, 8...) inférieurs ou égaux à n et qui contient dans la deuxième ligne que des zéros.

Exemple: Pour n=50 la fonction va retourner la liste L suivante :

$$L=[[1,2,3,5,8,13,21,34],[0,0,0,0,0,0,0,0]]$$

Question 3:

Écrire une fonction d'entête **def coder(n)** qui permet de coder la valeur n selon le code de Fibonacci. Le code sera stocké dans une liste L qu'on retourne.

Exemple: Pour n=50 on obtient la liste suivante :

$$L=[[1,2,3,5,8,13,21,34],[0,0,1,0,0,1,0,1]]$$

Question 4: Ecrire une fonction d'entête : **def decoder(X)** qui retourne la valeur entière correspondante au code de Fibonacci donné dans la liste X.

Exemple :

Pour X=[0,0,1,0,0,1,0,1] l'appel de la fonction **decoder(X)** retourne 50

Exercice 5: Un météorologiste souhaite améliorer l'efficacité du traitement des données qu'il utilise (tableau de température moyenne mensuelle ci-dessous). Son objectif étant de réaliser des traitements statistiques, il décide d'utiliser un dictionnaire en Python.

	Janvier	...	Décembre
France	06	...	08
...	...	...	...
Australie	35	...	30

En imposant que tous les pays aient 12 températures mensuelles et qu'elles soient classées dans le même ordre, le météorologiste utilise la liste **Mois** permettant de fixer l'ordre des températures ainsi que le nom de chaque mois, et le dictionnaire

Température :

Temperature = {"Maroc" : [39, 28, 34, 43, 39, 34, 29, 34, 25, 26, 30, 34],
"Espagne": [28, 25, 34, 34, 34, 32, 22, 25, 27, 34, 24, 24] ..., "France" : [26, 20, 24, 31, 33, 29, 34, 33, 33, 32, 34, 28] ,.....}

mois = ["janvier", "février", "mars", "avril", "mai", "juin", "juillet",
"août", "septembre", "octobre", "novembre", "décembre"]

] Question 1 Manipulation des données

Question 1.1. Donner le résultat de l'instruction :

```
print("La température au Maroc en février est:", Temperature["Maroc"][1])
```

Question 1.2 : Ecrire une fonction **AfficheMois(mois)**, ayant pour paramètre le nom d'un mois, et qui affiche pour chaque pays la température du mois fournis sous forme de chaîne de caractères : "janvier", "février", "mars",...

Question 1.3 : Afin de pouvoir modifier ses données, le météorologiste souhaite utiliser deux fonctions que vous devez écrire en python :

a) La Fonction **AjoutPays(pays,Temp)** qui prend comme paramètre le nom du pays ainsi que la liste des températures associées à ce pays et l'ajoute au dictionnaire Temperature.

Par exemple **AjoutPays(Bresil',[25,...23])**.

b) La Fonction **ModificationPaysMois(pays,mois,val_temp)** qui prend comme paramètre d'entrée le nom du pays, le nom du mois, la valeur de la nouvelle température, la liste Mois et comme paramètre d'entrée sortie Temperature. Cette fonction doit modifier la valeur de la température du pays pour le mois indiqué.

Par exemple **ModificationPaysMois("France","Janvier",27)** doit modifier la valeur de la température en France au mois de Janvier (ici 27 remplaçant la valeur 26).

Question 2 : Calcul statistique

Module: Python pour les sciences de données

Question 2.1 : Donner la fonction **MoyennePays(pays)** qui pour un pays donné en paramètre d'entrée donne la moyenne annuelle des températures.

Question 2.2 : Donner la fonction **moyenneMois(mois)** qui pour un mois donné (par exemple Mars) donne la moyenne générale des températures de tous les pays.

Question 2.3 : Donner la fonction **PaysAyantMoyenneMax ()** qui donne le pays dont la moyenne annuelle des températures est la plus élevée.

Exercice 6: DETERMINATION DES ITINERAIRES

Présentation. Dans le monde actuel, il est très utile de pouvoir localiser géographiquement des personnes ou des objets aussi bien pour des besoins privés que professionnels.

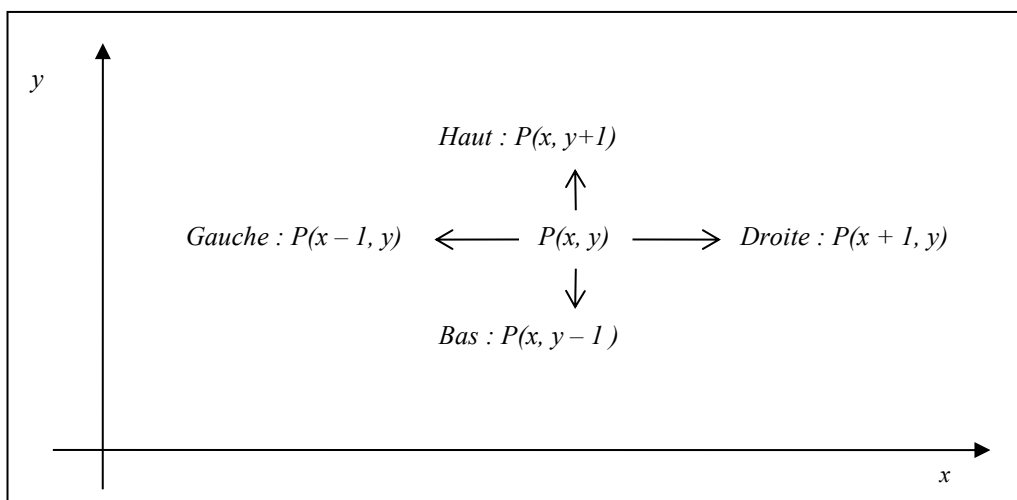
Au niveau professionnel, la géo-localisation est très utilisée notamment pour la navigation routière mais aussi dans plusieurs autres domaines (Transport, logistique, énergie, ..) favorisant ainsi un gain de productivité et une sécurité accrue.

Des systèmes de géo-localisation – tels que GPS – permettent le repérage de la position géographique exacte de leurs usagers, ainsi que le calcul d'itinéraires.

C'est dans cette optique que s'inscrit le problème suivant concernant des algorithmes de détermination de chemins et d'itinéraires entre les points d'un plan.

Notations.

- Soit P un plan, on notera $P(x, y)$, un point de coordonnées x et y dans ce plan représenté par un tuple (x, y) .
- Soit N une variable de type **int** strictement positive à laquelle on donnera la valeur 10 ($N = 10$) dans tout le problème. On notera $P(N)$, l'ensemble des points du plan P ayant des coordonnées x et y entières positives telles que $(0 \leq x < N)$ et $(0 \leq y < N)$.
- Soient A et B deux points distincts de $P(N)$. on appelle **chemin de A vers B**, une liste C , non vide et ordonnée de points distincts (tous différents) de $P(N)$ tel que :
 $C = [(x_A, y_A), (x_1, y_1), \dots, (x_i, y_i), \dots, (x_B, y_B)]$



Chaque point $P(x, y)$ de ce chemin est relié au point suivant $P(s, t)$ par l'une des 4 relations suivantes :

Module: Python pour les sciences de données

- a. Gauche :** $P(s, t)$ est à gauche de $P(x, y)$. c'est-à-dire : $s == x - 1$ et $t == y$
- b. Droite :** $P(s, t)$ est à droite de $P(x, y)$. c'est-à-dire : $s == x + 1$ et $t == y$
- c. Haut :** $P(s, t)$ est au dessus de $P(x, y)$. c'est-à-dire : $s == x$ et $t == y + 1$
- d. Bas :** $P(s, t)$ est en dessous de $P(x, y)$. c'est-à-dire : $s == x$ et $t == y - 1$

Exemples de chemins. Soient $A(2, 3)$ et $B(5, 5)$ deux points de $P(10)$:

$C1 = [(2, 3), (3, 3), (4, 3), (4, 4), (4, 5), (5, 5)]$

$C2 = [(2, 3), (2, 4), (3, 4), (3, 5), (4, 5), (5, 5)]$

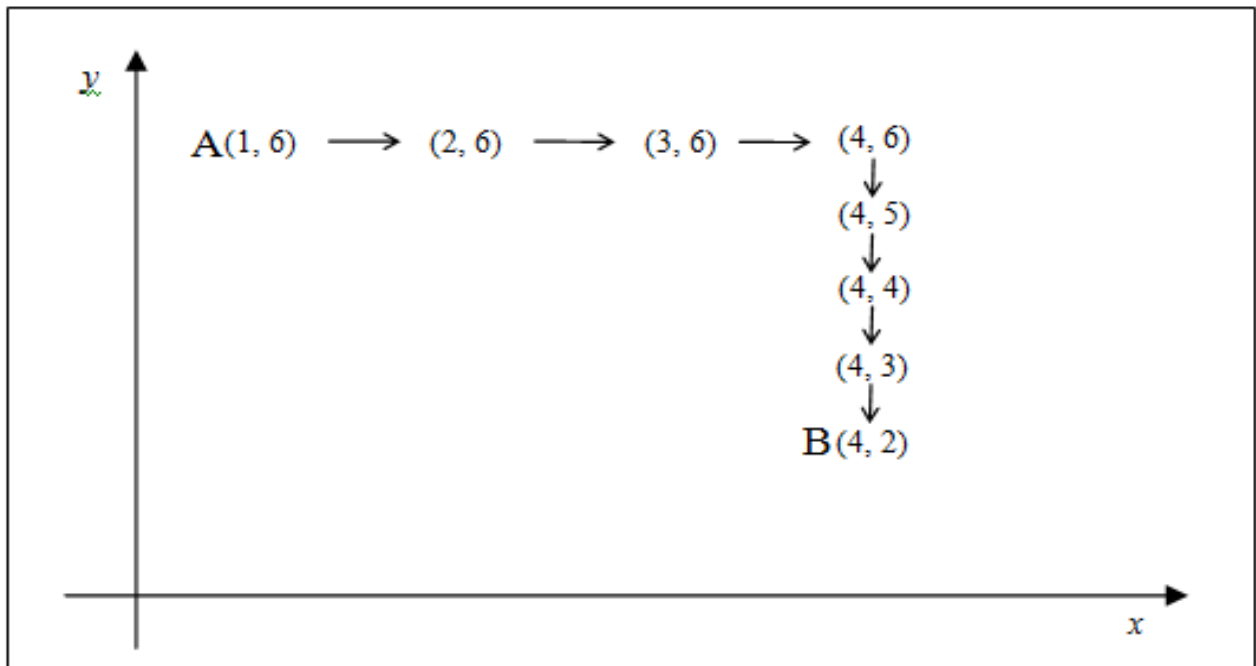
Les listes $C1$ et $C2$ représentent 2 chemins différents de A vers B .

Construction de chemins :

Chemin horizontal puis vertical. Soient A et B deux points de $P(N)$. On appelle **CheminHV** de A vers B le chemin construit de telle sorte à se déplacer à partir de A horizontalement (soit à gauche, soit à droite) jusqu'à arriver au point ayant la même abscisse que B , puis se déplacer verticalement (soit en haut, soit en bas) jusqu'à arriver au point B .

Exemple. $N = 10$. Soient $A = (1, 6)$ et $B = (4, 2)$ de $P(10)$, alors **CheminHV** de A vers B est :

$C = [(1, 6), (2, 6), (3, 6), (4, 6), (4, 5), (4, 4), (4, 3), (4, 2)]$



Question 1. Ecrire une fonction *cheminHV(A, B)* qui crée et retourne la liste *HVAB* de tuples représentant le cheminHV de A vers B. Dans ce cas *HVAB[0]* contiendra le tuple (x_A, y_A) . *HVAB[-1]* contiendra le tuple (x_B, y_B) .

Distance d'un chemin et chemin minimal. Soit C un chemin reliant deux points A et B. on appelle distance du chemin C, le nombre de points du chemin C différents du point A.

Exemple. Soient A= (2, 6) et B= (4, 3) deux points et C = [(2, 6), (2, 5), (2, 4), (2, 3), (3, 3), (4, 3)] un chemin de A vers B. La distance de C est de **5**.

On considère maintenant plusieurs chemins différents et tous menant de A vers B. On représente ces chemins par un dictionnaire *DictChemins* où les clés seront des chaînes de caractères : C₁, C₂, .. C_N et les valeurs sont les listes représentant ces chemins.

Exemple. Pour les deux points A= (2,6) et B= (4, 3), *DictChemins* aura la représentation suivante :

DictChemins = { 'C1' : [(2, 6), (3, 6), (4, 6), (4, 5), (4, 4), (4, 3)] ,
 'C2' : [(2, 6), (2, 5), (2, 4), (2, 3), (2, 2), (3, 2), (3, 3), (4,3)] ,
 'C3' : [(2, 6), (2, 5), (2, 4), (3, 4), (3, 3), (4, 3)] ,
 'C4' : [(2, 6), (1, 6), (0, 6), (0, 5), (0, 4), (0, 3), (1,
 3), (2, 3), (3, 3), (4, 3)] , }

Avec C1 et C3 de distance 5, C3 de distance 7 et C4 de distance 9.

D'autre part la distance minimale reliant les 2 points A et B (du dictionnaire *DictChemins*) est 5.

Le chemin minimal est donc C1 = [(2, 6), (3, 6), (4, 6), (4, 5), (4, 4), (4, 3)].

Si on a plusieurs chemins minimaux, on prend le premier chemin minimal trouvé pendant le parcours du dictionnaire.

Question 2. Ecrire une fonction *distanceChemin(cle)* qui retourne la distance d'un chemin identifié par sa clé passée en paramètre.

Question 3. Ecrire une fonction *distanceMinimale(DictChm)* qui retourne la distance minimale de plusieurs chemins du dictionnaire *DictChm* passé en paramètre.

Question 4. Ecrire une fonction *cheminMinimal(DictChm)* qui retourne le chemin minimal des chemins du dictionnaire *DictChm* passé en paramètre.

Question 5. Ecrire une fonction *cheminRetour(C)* qui retourne le chemin de retour qui est le chemin inverse de C. Si C est un chemin de A vers B, la fonction retourne le chemin de B vers A.